# ASSIGNMENT 4

**NAME: JANANIPRIYA G**
**REG.NO: 192324041**
**SUBMISSION DATE:13/11/26**
**SLOT: A**
**SUBJECT:JAVA PROGRAMMING**
**COURSE CODE: CSA0903**

## TASK 1 – GUI DESIGN REVIEW

**Diagnose and Repair a Poor Event-Handling Design**

**Task 1.1 – Diagnose the Design (2 Marks)**

**The given fee-payment program has the following design deficiencies:**

| S.No. | Design Deficiency | Explanation and Possible Problem |
|---|---|---|
| 1 | Poor separation of interface and logic | Database operations and payment logic are written inside actionPerformed(). This makes the program difficult to test and maintain. |
| 2 | Incorrect String comparison | The code uses == to compare action commands. It should use .equals() because == compares object references rather than string contents. |
| 3 | No input validation | Empty registration numbers, invalid amounts and negative values are not checked. This may cause errors or incorrect payment records. |
| 4 | Database operation on the event-dispatch thread | Database operations may take time and make the application unresponsive. |
| 5 | Use of Thread.sleep() | The three-second delay blocks the GUI thread, causing the window to freeze temporarily. |
| 6 | No proper user feedback | The program only prints "Paid" in the console instead of displaying a payment confirmation in the window. |

| | | |
|---|---|---|
| 7 | Receipt button is not implemented | Clicking Receipt executes the else block and clears the fields instead of generating a receipt. |
| 8 | SQL injection risk | SQL values are directly combined into a query. PreparedStatement should be used to prevent SQL injection. |
| 9 | No exception handling | Database and interruption exceptions are not handled properly. |
| 10 | No proper resource management | The database connection and statement are not closed, which may cause resource leaks. |

**Task 1.2 – Repair the Design (3 Marks)**

**The improved design separates the graphical user interface from the payment logic using two classes:**

**1. FeeWindow**

- **Collects the registration number and payment amount.**
- **Handles button-click events.**
- **Displays payment confirmation or error messages.**

**2. FeeService**

- **Validates the input.**
- **Performs the payment operation.**
- **Handles database-related operations.**

**Class Structure**

**FeeWindow**

**|**

**| calls**

**v**

**FeeService**

**|**

**| performs**

**v**

Payment Database

Java Program

FeeService.java

```
public class FeeService {

   public String makePayment(String regNo, String amountText) {

      if (regNo == null || regNo.trim().isEmpty()) {
         return "Registration number is required";
      }

      double amount;

      try {
         amount = Double.parseDouble(amountText);
      } catch (NumberFormatException e) {
         return "Enter a valid amount";
      }

      if (amount <= 0) {
         return "Amount must be greater than zero";
      }

      // Payment processing logic
      return "Payment successful for " + regNo
            + ". Amount: " + amount;
   }
}
```

**FeeWindow.java**

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class FeeWindow extends JFrame {

  JTextField regNo = new JTextField(15);
  JTextField amount = new JTextField(15);

  JButton pay = new JButton("Pay");
  JButton clear = new JButton("Clear");
  JButton receipt = new JButton("Receipt");

  FeeService service = new FeeService();

  public FeeWindow() {

    setTitle("Student Fee Payment");
    setLayout(new FlowLayout());

    add(new JLabel("Registration Number:"));
    add(regNo);

    add(new JLabel("Amount:"));
    add(amount);

    add(pay);
```

```
add(clear);
add(receipt);

pay.addActionListener(new ActionListener() {
  public void actionPerformed(ActionEvent e) {

    String result = service.makePayment(
        regNo.getText(), amount.getText());

    JOptionPane.showMessageDialog(
        FeeWindow.this, result);
  }
});

clear.addActionListener(new ActionListener() {
  public void actionPerformed(ActionEvent e) {
    regNo.setText("");
    amount.setText("");
  }
});

receipt.addActionListener(new ActionListener() {
  public void actionPerformed(ActionEvent e) {

    JOptionPane.showMessageDialog(
        FeeWindow.this,
        "Receipt for Registration Number: "
        + regNo.getText()
```

```
                + "\nAmount: " + amount.getText());
        }
    });

    setSize(350, 250);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
  }

  public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new FeeWindow());
  }
}
```

**Note: This is a demonstration of separation and validation. The FeeService currently simulates successful payment; actual JDBC payment storage should be added using PreparedStatement, with database work performed in a background worker such as SwingWorker.**

**Task 1.3 – Design Defence (1 Mark)**

**The redesigned program is easier to test and maintain because the user interface and payment logic are separated into different classes. The FeeWindow class handles user interaction, while the FeeService class handles validation and payment processing. Changes to the payment logic can be made without modifying the interface. The service class can also be tested independently, reducing errors and improving code reusability.**

**Alternative Concept and Complexity Analysis**

**Instead of using one listener with an if-else chain, each button can have its own listener object.**

**Example:**

```
pay.addActionListener(e -> {
  // Payment operation
});
```

```
clear.addActionListener(e -> {
   regNo.setText("");
   amount.setText("");
});

receipt.addActionListener(e -> {
   // Receipt generation
});
```

**Comparison**

| Feature | Single Listener | Separate Listeners |
|---|---|---|
| Event handling | Uses if-else conditions | Each button has its own handler |
| Event dispatch | Up to k comparisons | Direct listener invocation |
| Time complexity | O(k) worst case | O(1) dispatch |
| Space complexity | O(1) additional listener objects | O(k) listener objects |
| Maintenance | Becomes complex as buttons increase | Easier to modify individual actions |
| Reusability | Limited | Individual handlers can be reused |

**Conclusion: Separate listeners improve readability, modularity and maintainability when an application contains multiple buttons.**

**TASK 3 – FROM SKETCH TO LAYOUT**

**Campus Library Search Screen**

**Task 3.1 – Choose the Layout Managers (3 Marks)**

**The Campus Library Search Screen requires different layout managers to arrange its components properly.**

| S.No. | Container | Layout Manager | Reason |
|---|---|---|---|
| 1 | Main JFrame | BorderLayout | Divides the window into top, centre and bottom regions. |
| 2 | Search Panel | FlowLayout | Arranges the search field, category and Search button in a row. |
| 3 | Bottom Panel | FlowLayout | Arranges Prev, Next and Reserve buttons horizontally. |
| 4 | Results Panel | BorderLayout | Allows the results component to occupy the available centre space. |
| 5 | Status Panel | FlowLayout | Displays the status message at the bottom. |

**Justification: BorderLayout allows the central results area to expand when the window is resized. FlowLayout keeps the search controls and navigation buttons arranged in a simple row.**

**Task 3.2 – Design the Nested Containers (2 Marks)**

**Component Hierarchy**

**JFrame – Campus Library Search**

**(BorderLayout)**

**|**

**+-- NORTH: JPanel – Title Panel**

**| +-- JLabel – Campus Library Search**

**|**

**+-- CENTER: JPanel – Main Panel**

**| (BorderLayout)**

**| |**

**| +-- NORTH: JPanel – Search Panel**

**| | (FlowLayout)**

**| | +-- JTextField – Search**

**| | +-- JComboBox – Category**

| | +-- JButton – Search

| |

| +-- CENTER: JPanel – Results Panel

| | (BorderLayout)

| | +-- JTable – Search Results

| |

| +-- SOUTH: JPanel – Navigation Panel

| (FlowLayout)

| +-- JButton – Prev

| +-- JButton – Next

| +-- JButton – Reserve

|

+-- SOUTH: JPanel – Status Panel

(FlowLayout)

+-- JLabel – Status Message

**Task 3.3 – Implement the Screen (3 Marks)**

**Java Program**

**File Name: LibrarySearch.java**

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class LibrarySearch extends JFrame {

  JTextField searchField;
  JComboBox<String> category;
  JButton searchButton, prevButton, nextButton, reserveButton;
```

```
JLabel status;
JTable table;

public LibrarySearch() {

  setTitle("Campus Library Search");
  setLayout(new BorderLayout(5, 5));

  // Title Panel
  JLabel title = new JLabel(
      "CAMPUS LIBRARY SEARCH",
      JLabel.CENTER);

  JPanel titlePanel = new JPanel(new FlowLayout());
  titlePanel.add(title);

  // Search Panel
  JPanel searchPanel = new JPanel(new FlowLayout());

  searchField = new JTextField(15);

  category = new JComboBox<>(
      new String[]{"All", "Books", "Journals", "Articles"});

  searchButton = new JButton("Search");

  searchPanel.add(searchField);
  searchPanel.add(category);
```

```
searchPanel.add(searchButton);

// Results Table
String[] columns = {"Book ID", "Title", "Category"};

Object[][] data = {
  {"101", "Java Programming", "Books"},
  {"102", "Data Structures", "Books"},
  {"103", "AI Research", "Articles"}
};

DefaultTableModel model =
      new DefaultTableModel(data, columns);

table = new JTable(model);

JScrollPane scrollPane = new JScrollPane(table);

JPanel resultsPanel = new JPanel(new BorderLayout());
resultsPanel.add(scrollPane, BorderLayout.CENTER);

// Navigation Panel
JPanel navigationPanel = new JPanel(new FlowLayout());

prevButton = new JButton("Prev");
nextButton = new JButton("Next");
reserveButton = new JButton("Reserve");
```

```
navigationPanel.add(prevButton);
navigationPanel.add(nextButton);
navigationPanel.add(reserveButton);

// Main Panel
JPanel mainPanel = new JPanel(new BorderLayout(5, 5));

mainPanel.add(searchPanel, BorderLayout.NORTH);
mainPanel.add(resultsPanel, BorderLayout.CENTER);
mainPanel.add(navigationPanel, BorderLayout.SOUTH);

// Status Panel
status = new JLabel("Ready to search");

JPanel statusPanel = new JPanel(new FlowLayout());
statusPanel.add(status);

// Add panels to Frame
add(titlePanel, BorderLayout.NORTH);
add(mainPanel, BorderLayout.CENTER);
add(statusPanel, BorderLayout.SOUTH);

// Search Event
searchButton.addActionListener(e -> {
  String keyword = searchField.getText().trim();

  if (keyword.isEmpty()) {
    status.setText("Enter a search keyword");
```

```
        } else {
            status.setText("Search completed: " + keyword);
        }
    });

    // Navigation Events
    prevButton.addActionListener(e ->
            status.setText("Previous page selected"));

    nextButton.addActionListener(e ->
            status.setText("Next page selected"));

    reserveButton.addActionListener(e -> {
        int row = table.getSelectedRow();

        if (row >= 0) {
            status.setText("Book reserved successfully");
        } else {
            status.setText("Select a book to reserve");
        }
    });

    setSize(650, 450);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setVisible(true);
}
```

```
  public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new LibrarySearch());
  }
}
```

**Expected Result**

**The program displays:**

- **A title at the top.**
- **A search field, category selection and Search button.**
- **A results table in the centre.**
- **Prev, Next and Reserve buttons at the bottom of the main panel.**
- **A status message at the bottom of the window.**

**Task 3.4 – Test the Resizing Behaviour (1 Mark)**

**Run the program and capture screenshots at the following three window sizes.**

| Test Case | Window Size | Expected Behaviour |
|---|---|---|
| Test 1 | 500 × 350 | Results area is displayed in a smaller space. |
| Test 2 | 650 × 450 | Results area expands to the available centre space. |
| Test 3 | 850 × 600 | Results area grows while the search and navigation rows remain arranged horizontally. |

**Observation: When the window size increases, the central results area expands automatically because it is placed in the CENTER region of BorderLayout. The search and navigation panels remain arranged using FlowLayout.**

**Alternative Concept and Complexity Analysis**

**Approach 1 – Nested BorderLayout and FlowLayout**

- **Uses multiple panels with different layout managers.**
- **Automatically adjusts components when the window is resized.**
- **Keeps the interface organized and readable.**

**Time Complexity: Approximately O(n) for arranging n components during a layout pass.**

**Space Complexity: O(n) for storing the components and container hierarchy.**

**Approach 2 – GridBagLayout**

- **Uses constraints to position components in rows and columns.**
- **Allows flexible resizing and component alignment.**
- **Requires more configuration than BorderLayout and FlowLayout.**

**Time Complexity: Approximately O(n) per layout pass for n components, although actual work depends on the layout constraints.**

**Space Complexity: O(n) for components and their layout constraints.**

**Approach 3 – Absolute Positioning**

- **Uses setLayout(null) and setBounds() to specify component positions manually.**
- **Requires manually changing component sizes and positions when the window is resized.**
- **Does not automatically adapt to different screen sizes.**

**Time Complexity: O(n) to position n components when manually recalculating their bounds.**

**Space Complexity: O(n) for component references and position information.**

**Comparison**

| Feature | Nested Layouts | GridBagLayout | Absolute Positioning |
|---|---|---|---|
| Resizing | Automatic | Flexible | Manual |
| Alignment | Simple | Advanced | Manual |
| Coding difficulty | Easy | Moderate | Easy initially |
| Maintenance | Easy | Moderate | Difficult |
| Time complexity | O(n) | Approximately O(n) | O(n) |
| Space complexity | O(n) | O(n) | O(n) |

**Conclusion: Nested BorderLayout and FlowLayout are suitable for the Campus Library Search Screen because they provide automatic resizing, simple component arrangement and easy maintenance.**

**TASK 4 – EVENT-HANDLING FORENSICS**

**Drawing Pad Application**

**Task 4.1 – Locate the Defect (2 Marks)**

**The given DrawPad program contains the following defects:**

**1. Incorrect method name**

**The program uses:**

**public void mouseclicked(MouseEvent e)**

**The correct method name is:**

**public void mouseClicked(MouseEvent e)**

**Java is case-sensitive. Therefore, the listener does not override the correct MouseAdapter method, and the click event is not handled.**

**2. Label is not updated**

**The program does not update the status label after a mouse click. Therefore, the click count remains 0.**

**3. Incorrect drawing approach**

**The program uses getGraphics() to draw directly on the window. These drawings are temporary and may disappear when the window is repainted.**

**Task 4.2 – Construct a Failing Interaction (2 Marks)**

**The following steps demonstrate the repainting problem:**

1. **Run the DrawPad application.**
2. **Click several places inside the window.**
3. **Dots appear at the clicked positions.**
4. **Minimize the window.**
5. **Restore the window.**

**Observation: The previously drawn dots disappear.**

**Reason: The getGraphics() method draws directly on the screen without storing the drawing information. When the window is minimized, restored or repainted, the previous drawings are not automatically recreated.**

**Task 4.3 – Repair the Program (2 Marks)**

**The corrected program stores every clicked point in an ArrayList. The paint() method redraws all stored points whenever the window needs repainting.**

**Complete Java Program**

**File Name: DrawPad.java**

```
import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.Point;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.ArrayList;
import java.util.List;

public class DrawPad extends Frame {

  private final List<Point> points = new ArrayList<>();
  private final Label status = new Label("Clicks: 0");

  public DrawPad() {

    super("Draw Pad");

    add(status, BorderLayout.SOUTH);

    addMouseListener(new MouseAdapter() {
```

```
        // TODO 1
        @Override
        public void mouseClicked(MouseEvent e) {

            points.add(e.getPoint());

            status.setText("Clicks: " + points.size());

            repaint();
        }
    });

    addWindowListener(new WindowAdapter() {

        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setSize(400, 300);
}

@Override
public void paint(Graphics g) {

    // TODO 2
```

```
    for (Point p : points) {


      g.fillOval(
          p.x - 3,
          p.y - 3,
          6,
          6
      );
    }
  }


  public static void main(String[] args) {
    new DrawPad().setVisible(true);
  }
}
```

**Explanation of TODO 1**

- **mouseClicked() handles the mouse-click event.**
- **points.add(e.getPoint()) stores the clicked position.**
- **status.setText() updates the click count.**
- **repaint() requests the window to redraw its contents.**

**Explanation of TODO 2**

- **The paint() method loops through the stored points.**
- **fillOval() draws a 6 × 6 filled circle.**
- **Subtracting 3 from the coordinates places the centre of each circle at the clicked position.**

**Expected Output**

**Initially:**

**Draw Pad**

**Clicks: 0**

**After three mouse clicks:**

**Draw Pad**

**Clicks: 3**

**Three dots appear at the clicked positions and remain visible after minimizing and restoring the window.**

**Task 4.4 – Prevent Recurrence (1 Mark)**

**The @Override annotation helps identify incorrect method names during compilation. If mouseclicked() is written instead of mouseClicked(), the compiler reports that the method does not override or implement a method from a supertype. Therefore, using @Override prevents accidental method-name mistakes and improves code reliability.**

**Complete the Code – TODO Answers**

**TODO 1:**

```
@Override
public void mouseClicked(MouseEvent e) {
   points.add(e.getPoint());
   status.setText("Clicks: " + points.size());
   repaint();
}
```

**TODO 2:**

```
for (Point p : points) {
   g.fillOval(p.x - 3, p.y - 3, 6, 6);
}
```

**Additional Experiment**

**Remove @Override and change the method name to mouseclicked.**

**Observation: The program may compile, but the mouse-click event will not execute the intended method because it is not overriding the correct MouseAdapter method.**

**When @Override is included, the compiler identifies the incorrect method name.**

**Alternative Concept and Complexity Analysis**

**Instead of storing individual points in an ArrayList, the program can use an off-screen BufferedImage to preserve the drawing.**

**Approach 1 – Using ArrayList of Points**

- **Store each mouse-click position in an ArrayList.**
- **Redraw all points inside the paint() method.**
- **Every repaint requires drawing all stored points.**

**Time Complexity: O(n) per repaint, where n is the number of clicks.**

**Space Complexity: O(n), because each point is stored separately.**

**Approach 2 – Using BufferedImage**

- **Create an off-screen image using BufferedImage.**
- **Draw each new dot directly onto the image when a click occurs.**
- **Copy the stored image to the screen inside paint().**

**Example:**

```
BufferedImage image =
   new BufferedImage(400, 300, BufferedImage.TYPE_INT_ARGB);


Graphics g = image.getGraphics();


g.fillOval(e.getX() - 3, e.getY() - 3, 6, 6);

repaint();
```

**Inside paint():**

```
g.drawImage(image, 0, 0, this);
```

**Time Complexity: O(1) to add a dot, excluding repaint overhead; O(width × height) for copying the image during a full repaint.**

**Space Complexity: O(width × height) pixels for the image.**

**Comparison**

| Feature | ArrayList Approach | BufferedImage Approach |
|---|---|---|
| Storage | Stores individual points | Stores complete drawing |
| Drawing | Redraws every point | Copies the stored image |
| Time per repaint | O(n) | O(width × height) |
| Space complexity | O(n) | O(width × height) |
| Implementation | Simple | Requires image initialization and graphics handling |
| Suitable for | Small drawings | Large or complex drawings |

**Conclusion: ArrayList is simple and suitable for storing a small number of points. BufferedImage is more efficient for repeatedly displaying complex drawings because the drawing is preserved as an image.**

**TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES**

**Currency Converter Application**

**Task 5.1 – Four Implementations (3 Marks)**

**The currency converter can handle button-click events using four different approaches.**

**(a) Frame Implements ActionListener**

**In this approach, the main JFrame class implements the ActionListener interface and overrides the actionPerformed() method.**

```
import java.awt.*;

import java.awt.event.*;

import javax.swing.*;


public class ConverterA extends JFrame implements ActionListener {
```

```
JTextField amount = new JTextField(10);
JComboBox<String> target =
    new JComboBox<>(new String[]{"USD", "EUR"});
JLabel result = new JLabel(" ");
JButton convert = new JButton("Convert");

public ConverterA() {

  setLayout(new FlowLayout());

  add(new JLabel("INR"));
  add(amount);
  add(target);
  add(convert);
  add(result);

  convert.addActionListener(this);

  addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
      dispose();
    }
  });

  setSize(300, 180);
  setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  setVisible(true);
}
```

```
    public void actionPerformed(ActionEvent e) {
        convertAmount();
    }

    void convertAmount() {
        try {
            double value = Double.parseDouble(amount.getText());
            double rate = target.getSelectedItem().equals("USD")
                    ? 0.012 : 0.011;

            result.setText(String.format("%.2f",
                    value * rate));

        } catch (NumberFormatException e) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        new ConverterA();
    }
}
```

**(b) Named Inner Class**

**A named inner class is created inside the main class and implements ActionListener.**

```
import java.awt.*;
import java.awt.event.*;
```

```
import javax.swing.*;

public class ConverterB extends JFrame {

   JTextField amount = new JTextField(10);
   JComboBox<String> target =
         new JComboBox<>(new String[]{"USD", "EUR"});
   JLabel result = new JLabel(" ");
   JButton convert = new JButton("Convert");

   public ConverterB() {

      setLayout(new FlowLayout());

      add(new JLabel("INR"));
      add(amount);
      add(target);
      add(convert);
      add(result);

      convert.addActionListener(new ConvertHandler());

      addWindowListener(new WindowAdapter() {
         public void windowClosing(WindowEvent e) {
            dispose();
         }
      });
```

```
    setSize(300, 180);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
}

class ConvertHandler implements ActionListener {

    public void actionPerformed(ActionEvent e) {
        try {
            double value =
                Double.parseDouble(amount.getText());

            double rate =
                target.getSelectedItem().equals("USD")
                ? 0.012 : 0.011;

            result.setText(String.format("%.2f",
                value * rate));

        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }
}

public static void main(String[] args) {
    new ConverterB();
}
```

**}**

**(c) Anonymous Inner Class**

**An anonymous inner class implements ActionListener directly while registering the event listener.**

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ConverterC extends JFrame {

  JTextField amount = new JTextField(10);
  JComboBox<String> target =
      new JComboBox<>(new String[]{"USD", "EUR"});
  JLabel result = new JLabel(" ");
  JButton convert = new JButton("Convert");

  public ConverterC() {

    setLayout(new FlowLayout());

    add(new JLabel("INR"));
    add(amount);
    add(target);
    add(convert);
    add(result);

    convert.addActionListener(new ActionListener() {
```

```
    public void actionPerformed(ActionEvent e) {
      try {
        double value =
            Double.parseDouble(amount.getText());

        double rate =
            target.getSelectedItem().equals("USD")
            ? 0.012 : 0.011;

        result.setText(String.format("%.2f",
            value * rate));

      } catch (NumberFormatException ex) {
        result.setText("Invalid amount");
      }
    }
  });

  addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
      dispose();
    }
  });

  setSize(300, 180);
  setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  setVisible(true);
}
```

```
  public static void main(String[] args) {
    new ConverterC();
  }
}
```

**(d) Lambda Expression**

**A lambda expression provides a shorter way to implement the ActionListener interface.**

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ConverterD extends JFrame {

  JTextField amount = new JTextField(10);
  JComboBox<String> target =
      new JComboBox<>(new String[]{"USD", "EUR"});
  JLabel result = new JLabel(" ");
  JButton convert = new JButton("Convert");

  public ConverterD() {

    setLayout(new FlowLayout());

    add(new JLabel("INR"));
    add(amount);
    add(target);
    add(convert);
```

```
    add(result);

    convert.addActionListener(e -> convertAmount());

    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setSize(300, 180);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
}

void convertAmount() {
    try {
        double value = Double.parseDouble(amount.getText());

        double rate = target.getSelectedItem().equals("USD")
                ? 0.012 : 0.011;

        result.setText(String.format("%.2f",
                value * rate));

    } catch (NumberFormatException e) {
        result.setText("Invalid amount");
    }
```

```
  }

  public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new ConverterD());
  }
}
```

**Task 5.2 – Change the Requirement (2 Marks)**

**The converter must now perform conversion when the user presses Enter in the amount field and display the result in a dialog box.**

**The same conversion logic can be reused by both the button and the text field.**

**Modified Event Handling**

**For all four implementations, add the following event registration:**

**amount.addActionListener(e -> convertAmount());**

**For the frame-based version, use:**

**amount.addActionListener(this);**

**The named inner class and anonymous inner class versions can use a shared handler object or register their respective handler again.**

**Modified Conversion Method**

**Replace the previous convertAmount() method with:**

```
void convertAmount() {

  try {
    double value = Double.parseDouble(amount.getText());

    double rate = target.getSelectedItem().equals("USD")
        ? 0.012 : 0.011;

    double converted = value * rate;
```

```
        JOptionPane.showMessageDialog(
            this,
            String.format("Converted Amount: %.2f",
                converted));

    } catch (NumberFormatException e) {

        JOptionPane.showMessageDialog(
            this,
            "Invalid amount",
            "Input Error",
            JOptionPane.ERROR_MESSAGE);
    }
}
```

**Changes Made:**

| Implementation | Modification Required |
|---|---|
| Frame implements ActionListener | Register the text field with the frame and handle both sources in actionPerformed(). |
| Named inner class | Register the same named handler with the text field. |
| Anonymous inner class | Add another listener or extract a shared conversion method. |
| Lambda expression | Add another lambda or reuse the convertAmount() method. |

**Task 5.3 – Compare the Styles (2 Marks)**

| Feature | Frame Listener | Named Inner Class | Anonymous Inner Class | Lambda |
|---|---|---|---|---|
| Readability | Moderate | Good | Good for short logic | Very good for short logic |

| | | | | |
|---|---|---|---|---|
| **Access to frame fields** | **Direct** | **Direct** | **Direct** | **Direct** |
| **Reusability** | **Limited** | **High** | **Limited** | **Depends on implementation** |
| **Code length** | **Moderate** | **Longer** | **Moderate** | **Short** |
| **Maintenance** | **Can become complex** | **Easy for larger handlers** | **Suitable for small handlers** | **Suitable for simple actions** |
| **Separate handler class** | **No** | **Yes, inner class** | **No named class** | **No named class** |

**Preferred Approach: For a large application, I would prefer named inner classes or separate handler classes. They improve code organization, make event-handling logic easier to maintain and allow handlers to be reused. Lambda expressions are more suitable for short and simple event operations.**

**Complete the Code – TODO Answers**

**The following is the completed lambda version given in the assignment.**

**File Name: Converter.java**

```
import java.awt.FlowLayout;

import javax.swing.JButton;

import javax.swing.JComboBox;

import javax.swing.JFrame;

import javax.swing.JLabel;

import javax.swing.JTextField;

import javax.swing.JOptionPane;

import javax.swing.SwingUtilities;


public class Converter extends JFrame {


   private final JTextField amount = new JTextField(10);
```

```
private final JComboBox<String> target =
      new JComboBox<>(new String[]{"USD", "EUR"});

private final JLabel result = new JLabel(" ");

private static double rate(String currency) {
   return currency.equals("USD") ? 0.012 : 0.011;
}

public Converter() {

   super("Currency Converter");

   setLayout(new FlowLayout());

   JButton convertButton = new JButton("Convert");

   add(new JLabel("INR"));
   add(amount);
   add(target);
   add(convertButton);
   add(result);

   // TODO 1
   convertButton.addActionListener(e -> convert());
   amount.addActionListener(e -> convert());

   setDefaultCloseOperation(EXIT_ON_CLOSE);
```

```
    pack();
}

private void convert() {

    // TODO 2
    try {
        double value = Double.parseDouble(amount.getText());

        if (!Double.isFinite(value) || value < 0) {
            throw new NumberFormatException();
        }

        double converted =
                value * rate((String) target.getSelectedItem());

        String message = String.format(
                "Converted Amount: %.2f", converted);

        result.setText(String.format("%.2f", converted));

        JOptionPane.showMessageDialog(this, message);

    } catch (NumberFormatException e) {

        result.setText("Invalid amount");
```

```
        JOptionPane.showMessageDialog(
                this,
                "Invalid amount",
                "Input Error",
                JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(
                () -> new Converter().setVisible(true));
    }
}
```

**TODO 1 Answer**

```
convertButton.addActionListener(e -> convert());
amount.addActionListener(e -> convert());
```

**Both the Convert button and Enter key call the same conversion method.**

**TODO 2 Answer**

**The convert() method:**

1. **Reads the entered amount.**
2. **Converts the input into a double value.**
3. **Checks whether the value is valid and non-negative.**
4. **Calculates the converted currency using the selected rate.**
5. **Displays the result in a dialog box.**
6. **Displays an error message for invalid input.**

**Expected Output**

**Valid Input:**

**INR: 1000**

**Target: USD**

**Converted Amount: 12.00**

**Invalid Input:**

**INR: abc**

**Invalid amount**

**Alternative Concept and Complexity Analysis**

**Approach 1 – Lambda Expression**

**The Convert button and Enter key can call the same conversion method using lambda expressions.**

```
convertButton.addActionListener(e -> convert());

amount.addActionListener(e -> convert());
```

**Approach 2 – Method Reference**

**A method reference can directly connect the event to the conversion method.**

```
convertButton.addActionListener(e -> convert());
```

**Because convert() has no parameters, it cannot be directly used as an ActionListener method reference. A compatible method reference can be created using a helper method that accepts an ActionEvent:**

```
private void handleConvert(ActionEvent e) {
   convert();
}

convertButton.addActionListener(this::handleConvert);
```

**Approach 3 – Shared Action Object**

**A shared Swing Action can be used by a button, menu item and keyboard shortcut.**

```
Action convertAction = new AbstractAction("Convert") {
   public void actionPerformed(ActionEvent e) {
      convert();
```

```
    }
};

convertButton.setAction(convertAction);
```

**The same action can be assigned to a menu item or keyboard shortcut.**

**Comparison**

| Feature | Separate Lambda | Method Reference | Shared Action |
|---|---|---|---|
| Code length | Short | Very short | Moderate |
| Reusability | Through shared method | Through referenced method | High |
| Multiple triggers | Separate registrations | Separate registrations | Same Action can be reused |
| Listener/action | | | |

## TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES

**Currency Converter Application**

**Task 5.1 – Four Implementations (3 Marks)**

**The currency converter can handle button-click events using four different approaches.**

**(a) Frame Implements ActionListener**

**In this approach, the main JFrame class implements the ActionListener interface and overrides the actionPerformed() method.**

```
import java.awt.*;

import java.awt.event.*;

import javax.swing.*;


public class ConverterA extends JFrame implements ActionListener {
```

```
JTextField amount = new JTextField(10);
JComboBox<String> target =
    new JComboBox<>(new String[]{"USD", "EUR"});
JLabel result = new JLabel(" ");
JButton convert = new JButton("Convert");

public ConverterA() {

  setLayout(new FlowLayout());

  add(new JLabel("INR"));
  add(amount);
  add(target);
  add(convert);
  add(result);

  convert.addActionListener(this);

  addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
      dispose();
    }
  });

  setSize(300, 180);
  setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  setVisible(true);
```

```
    }

    public void actionPerformed(ActionEvent e) {
        convertAmount();
    }

    void convertAmount() {
        try {
            double value = Double.parseDouble(amount.getText());
            double rate = target.getSelectedItem().equals("USD")
                    ? 0.012 : 0.011;

            result.setText(String.format("%.2f",
                    value * rate));

        } catch (NumberFormatException e) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        new ConverterA();
    }
}
```

**(b) Named Inner Class**

**A named inner class is created inside the main class and implements ActionListener.**

```
import java.awt.*;
```

```
import java.awt.event.*;
import javax.swing.*;

public class ConverterB extends JFrame {

  JTextField amount = new JTextField(10);
  JComboBox<String> target =
      new JComboBox<>(new String[]{"USD", "EUR"});
  JLabel result = new JLabel(" ");
  JButton convert = new JButton("Convert");

  public ConverterB() {

    setLayout(new FlowLayout());

    add(new JLabel("INR"));
    add(amount);
    add(target);
    add(convert);
    add(result);

    convert.addActionListener(new ConvertHandler());

    addWindowListener(new WindowAdapter() {
      public void windowClosing(WindowEvent e) {
        dispose();
      }
    });
```

```
    setSize(300, 180);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
}

class ConvertHandler implements ActionListener {

    public void actionPerformed(ActionEvent e) {
        try {
            double value =
                Double.parseDouble(amount.getText());

            double rate =
                target.getSelectedItem().equals("USD")
                ? 0.012 : 0.011;

            result.setText(String.format("%.2f",
                value * rate));

        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }
}

public static void main(String[] args) {
    new ConverterB();
```

```
    }
}
```

**(c) Anonymous Inner Class**

**An anonymous inner class implements ActionListener directly while registering the event listener.**

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ConverterC extends JFrame {

    JTextField amount = new JTextField(10);
    JComboBox<String> target =
        new JComboBox<>(new String[]{"USD", "EUR"});
    JLabel result = new JLabel(" ");
    JButton convert = new JButton("Convert");

    public ConverterC() {

        setLayout(new FlowLayout());

        add(new JLabel("INR"));
        add(amount);
        add(target);
        add(convert);
        add(result);

        convert.addActionListener(new ActionListener() {
```

```
  public void actionPerformed(ActionEvent e) {
    try {
      double value =
          Double.parseDouble(amount.getText());

      double rate =
          target.getSelectedItem().equals("USD")
          ? 0.012 : 0.011;

      result.setText(String.format("%.2f",
          value * rate));

    } catch (NumberFormatException ex) {
      result.setText("Invalid amount");
    }
  }
});

addWindowListener(new WindowAdapter() {
  public void windowClosing(WindowEvent e) {
    dispose();
  }
});

setSize(300, 180);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setVisible(true);
```

```
  }

  public static void main(String[] args) {
    new ConverterC();
  }
}
```

**(d) Lambda Expression**

**A lambda expression provides a shorter way to implement the ActionListener interface.**

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ConverterD extends JFrame {

  JTextField amount = new JTextField(10);
  JComboBox<String> target =
      new JComboBox<>(new String[]{"USD", "EUR"});
  JLabel result = new JLabel(" ");
  JButton convert = new JButton("Convert");

  public ConverterD() {

    setLayout(new FlowLayout());

    add(new JLabel("INR"));
    add(amount);
    add(target);
```

```
    add(convert);
    add(result);

    convert.addActionListener(e -> convertAmount());

    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setSize(300, 180);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
}

void convertAmount() {
    try {
        double value = Double.parseDouble(amount.getText());

        double rate = target.getSelectedItem().equals("USD")
                ? 0.012 : 0.011;

        result.setText(String.format("%.2f",
                value * rate));

    } catch (NumberFormatException e) {
        result.setText("Invalid amount");
```

```
    }
  }

  public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new ConverterD());
  }
}
```

**Task 5.2 – Change the Requirement (2 Marks)**

**The converter must now perform conversion when the user presses Enter in the amount field and display the result in a dialog box.**

**The same conversion logic can be reused by both the button and the text field.**

**Modified Event Handling**

**For all four implementations, add the following event registration:**

**amount.addActionListener(e -> convertAmount());**

**For the frame-based version, use:**

**amount.addActionListener(this);**

**The named inner class and anonymous inner class versions can use a shared handler object or register their respective handler again.**

**Modified Conversion Method**

**Replace the previous convertAmount() method with:**

```
void convertAmount() {

  try {
    double value = Double.parseDouble(amount.getText());

    double rate = target.getSelectedItem().equals("USD")
        ? 0.012 : 0.011;

    double converted = value * rate;
```

```
        JOptionPane.showMessageDialog(
            this,
            String.format("Converted Amount: %.2f",
                converted));

    } catch (NumberFormatException e) {

        JOptionPane.showMessageDialog(
            this,
            "Invalid amount",
            "Input Error",
            JOptionPane.ERROR_MESSAGE);
    }
}
```

**Changes Made:**

| Implementation | Modification Required |
|---|---|
| Frame implements ActionListener | Register the text field with the frame and handle both sources in actionPerformed(). |
| Named inner class | Register the same named handler with the text field. |
| Anonymous inner class | Add another listener or extract a shared conversion method. |
| Lambda expression | Add another lambda or reuse the convertAmount() method. |

**Task 5.3 – Compare the Styles (2 Marks)**

| Feature | Frame Listener | Named Inner Class | Anonymous Inner Class | Lambda |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| **Readability** | **Moderate** | **Good** | **Good for short logic** | **Very good for short logic** |
| **Access to frame fields** | **Direct** | **Direct** | **Direct** | **Direct** |
| **Reusability** | **Limited** | **High** | **Limited** | **Depends on implementation** |
| **Code length** | **Moderate** | **Longer** | **Moderate** | **Short** |
| **Maintenance** | **Can become complex** | **Easy for larger handlers** | **Suitable for small handlers** | **Suitable for simple actions** |
| **Separate handler class** | **No** | **Yes, inner class** | **No named class** | **No named class** |

**Preferred Approach: For a large application, I would prefer named inner classes or separate handler classes. They improve code organization, make event-handling logic easier to maintain and allow handlers to be reused. Lambda expressions are more suitable for short and simple event operations.**

**Complete the Code – TODO Answers**

**The following is the completed lambda version given in the assignment.**

**File Name: Converter.java**

```
import java.awt.FlowLayout;

import javax.swing.JButton;

import javax.swing.JComboBox;

import javax.swing.JFrame;

import javax.swing.JLabel;

import javax.swing.JTextField;

import javax.swing.JOptionPane;

import javax.swing.SwingUtilities;


public class Converter extends JFrame {
```

```
private final JTextField amount = new JTextField(10);

private final JComboBox<String> target =
      new JComboBox<>(new String[]{"USD", "EUR"});

private final JLabel result = new JLabel(" ");

private static double rate(String currency) {
   return currency.equals("USD") ? 0.012 : 0.011;
}

public Converter() {

   super("Currency Converter");

   setLayout(new FlowLayout());

   JButton convertButton = new JButton("Convert");

   add(new JLabel("INR"));
   add(amount);
   add(target);
   add(convertButton);
   add(result);

   // TODO 1
   convertButton.addActionListener(e -> convert());
   amount.addActionListener(e -> convert());
```

```
    setDefaultCloseOperation(EXIT_ON_CLOSE);

    pack();
}

private void convert() {

    // TODO 2
    try {
        double value = Double.parseDouble(amount.getText());

        if (!Double.isFinite(value) || value < 0) {
            throw new NumberFormatException();
        }

        double converted =
                value * rate((String) target.getSelectedItem());

        String message = String.format(
                "Converted Amount: %.2f", converted);

        result.setText(String.format("%.2f", converted));

        JOptionPane.showMessageDialog(this, message);

    } catch (NumberFormatException e) {
```

```
            result.setText("Invalid amount");


            JOptionPane.showMessageDialog(
                this,
                "Invalid amount",
                "Input Error",
                JOptionPane.ERROR_MESSAGE);
        }
    }


    public static void main(String[] args) {
        SwingUtilities.invokeLater(
            () -> new Converter().setVisible(true));
    }
}
```

**TODO 1 Answer**

```
convertButton.addActionListener(e -> convert());
amount.addActionListener(e -> convert());
```

**Both the Convert button and Enter key call the same conversion method.**

**TODO 2 Answer**

**The convert() method:**

1. **Reads the entered amount.**
2. **Converts the input into a double value.**
3. **Checks whether the value is valid and non-negative.**
4. **Calculates the converted currency using the selected rate.**
5. **Displays the result in a dialog box.**
6. **Displays an error message for invalid input.**

**Expected Output**

**Valid Input:**

**INR: 1000**

**Target: USD**

**Converted Amount: 12.00**

**Invalid Input:**

**INR: abc**

**Invalid amount**

**Alternative Concept and Complexity Analysis**

**Approach 1 – Lambda Expression**

**The Convert button and Enter key can call the same conversion method using lambda expressions.**

```
convertButton.addActionListener(e -> convert());

amount.addActionListener(e -> convert());
```

**Approach 2 – Method Reference**

**A method reference can directly connect the event to the conversion method.**

```
convertButton.addActionListener(e -> convert());
```

**Because convert() has no parameters, it cannot be directly used as an ActionListener method reference. A compatible method reference can be created using a helper method that accepts an ActionEvent:**

```
private void handleConvert(ActionEvent e) {
   convert();
}

convertButton.addActionListener(this::handleConvert);
```

**Approach 3 – Shared Action Object**

**A shared Swing Action can be used by a button, menu item and keyboard shortcut.**

```
Action convertAction = new AbstractAction("Convert") {
```

```
    public void actionPerformed(ActionEvent e) {
        convert();
    }
};

convertButton.setAction(convertAction);
```

**The same action can be assigned to a menu item or keyboard shortcut.**

**Comparison**

| Feature | Separate Lambda | Method Reference | Shared Action |
|---|---|---|---|
| Code length | Short | Very short | Moderate |
| Reusability | Through shared method | Through referenced method | High |
| Multiple triggers | Separate registrations | Separate registrations | Same Action can be reused |
| Listener/action management | Individual registrations | Individual registrations | Centralized |
| Suitable for | Simple events | Existing handler methods | Large applications |

**Time Complexity: Currency conversion requires a constant number of arithmetic and validation operations, so the processing time is O(1) per event.**

**Space Complexity: Each independently created listener or action requires O(1) space. For k independently registered listener objects, the total listener storage is O(k). Reusing one shared Action reduces the number of separate action objects to O(1), excluding component references.**

**Conclusion: Lambda expressions are suitable for simple event handling. Shared Action objects are more useful when the same operation must be triggered from multiple interface components.**